PROCEDIMIENTO DE IMPLEMENTACIÓN DEL SOFTWARE

LAVA RÁPIDO VEHICULAR

Plataforma para la gestión de reservas, pagos y servicios de autolavado

José Stiven Perdomo Martínez

Santiago Gordo Pérez

Servicio Nacional de Aprendizaje – SENA

Tecnólogo en Análisis y Desarrollo de Software

Ficha 3145555

2026

Contenido

1. Introducción, propósito y alcance

2. Arquitectura y versiones del proyecto

3. Ambientes, prerrequisitos y responsabilidades

4. Preparación y control de versiones

5. Base de datos PostgreSQL y Liquibase

6. Backend Spring Boot y seguridad

7. Integraciones: Wompi, correo y notificaciones

8. Panel administrativo React

9. Aplicación móvil Expo/React Native

10. Procedimiento de despliegue y publicación

11. Verificación posterior y evidencias

12. Gestión de incidencias y reversión

13. Lista de comprobación de entrega

14. Conclusiones y referencias

1. Introducción, propósito y alcance

Lava Rápido Vehicular integra una aplicación móvil para clientes y operadores, un panel administrativo web, una API Spring Boot y una base de datos PostgreSQL. El procedimiento establece el orden de preparación, configuración, ejecución, publicación y comprobación de sus componentes. Se documenta el diseño de implementación y el estado identificado en los repositorios al 4 de noviembre de 2026, diferenciándolo de la operación efectivamente confirmada en un ambiente externo.

1.1 Propósito

Proporcionar instrucciones reproducibles para instalar e integrar los componentes, proteger la información sensible, ejecutar comprobaciones técnicas y conservar evidencias de liberación con trazabilidad de versión, ambiente y resultado.

1.2 Alcance

Comprende desarrollo local, ambiente controlado de pruebas y despliegue remoto; configuración de frontend móvil y web, backend y esquema PostgreSQL; seguridad JWT; pago Wompi; recuperación por correo y notificaciones. No comprende seguimiento GPS de operadores, optimización de rutas, inventario físico ni contabilidad detallada.

1.3 Criterio de interpretación

Los comandos son procedimientos propuestos para ejecutar por personal autorizado. Una configuración presente en el repositorio no equivale por sí sola a una publicación realizada. Las comprobaciones de octubre y las versiones declaradas en noviembre se identifican con su fecha y alcance; no se adjudican resultados de octubre a una versión posterior.

2. Arquitectura y versiones del proyecto

La solución sigue el enfoque cliente-servidor. El backend es una aplicación Spring Boot única, organizada por dominios y capas Controller–Service–Repository; no se trata de microservicios independientes. Los clientes consumen API REST y no acceden directamente a PostgreSQL.

Capa	Tecnologías declaradas (04/11/2026)	Función
Móvil	Expo ~58.0.20; React Native 0.88.0; TypeScript ~6.0.3	Experiencia de cliente y operador

Administración	React 19; Vite 6; React Router 7; Zustand 5	Gestión administrativa por navegador
API	Java 17; Spring Boot 4.0.9; Maven Wrapper	Reglas, permisos JWT y API REST
Datos	PostgreSQL 16; Liquibase 5.0.2 en Compose	Persistencia y migraciones
Servicios externos	Wompi; SMTP; Expo Push	Pagos, recuperación y entregas condicionadas

La base declarada contiene 18 tablas de aplicación y 24 changesets activos en los archivos revisados. Esto no demuestra que cada changeSet esté aplicado en la instancia desplegada.

2.1 Comunicación entre componentes

```
Aplicación móvil / Panel web  -->  API REST Spring Boot
API --> Repositorios JPA --> PostgreSQL
API --> Wompi / SMTP / Expo Push (según configuración)
```

La iniciación del pago puede utilizar el widget del proveedor en el cliente, pero la aprobación depende de verificación del backend mediante evidencia válida de Wompi; no de una redirección visual.

3. Ambientes, prerrequisitos y responsabilidades

Ambiente	Finalidad	Requisito de control
Desarrollo	Programación y ejecución local	Variables separadas, datos ficticios, Git identificado
Pruebas/QA	Integración y regresión con servicios de prueba	Base aislada, Wompi sandbox, evidencias depuradas
Producción	Servicio publicado y accesible por HTTPS	Versiones aprobadas, respaldo, dominios, permisos y monitoreo

3.1 Prerrequisitos del equipo

- Java 17 compatible con el proyecto; Maven Wrapper provisto por el repositorio (sin imponer una versión Maven global).
- Node.js y npm compatibles con los manifests y lockfiles; Docker y Docker Compose para la base local y, cuando proceda, backend.
- Git para inventario de ramas/commits; conectividad de red y acceso autorizado a las cuentas de hosting y proveedores.

- Variables de entorno configuradas fuera de documentos compartidos. No publicar secretos JWT, de PostgreSQL, correo o Wompi.

3.2 Responsabilidad por actividad

Actividad	Rol responsable sugerido	Evidencia a conservar
Versionado y construcción	Equipo de desarrollo	Commit, log y resultado de build
Migraciones y respaldo	Responsable de datos/backend	Raíz Liquibase, checksums, respaldo
Pruebas e integración	Equipo de desarrollo y pruebas	Casos, resultados y defectos
Publicación y monitoreo	Responsable de despliegue	URL, estado, fecha y revisión

4. Preparación y control de versiones

Antes de instalar, identificar la copia real de los proyectos. Las siguientes son rutas locales de referencia, no ubicaciones obligatorias para otros equipos:

```
C:\Frontend-lavarapido\lava-rapido-mobile
C:\Frontend-lavarapido\lava-rapido-admin
C:\backend\backend-vehicular
C:\lavarapido-base-datos
```

1. Comprobar la raíz de cada repositorio y anotar el commit seleccionado para la entrega.
2. Comprobar cambios pendientes antes de compilar. No cambiar ramas ni sobrescribir archivos sin aprobación.
3. Registrar versiones desde package.json, pom.xml, Docker Compose y archivos de configuración.
4. Separar variables por desarrollo, pruebas y producción. Nunca incluir sus valores sensibles en el informe.
5. Confirmar puertos, hostnames, CORS, base URL de API y HTTPS para los clientes publicados.

```
git status --short
git branch --show-current
git rev-parse HEAD
java -version
node --version
npm --version
docker compose version
```

Como referencia documental del 4/11/2026, el backend estaba en develop, el frontend en HU-88-dev y la base de datos en main. Estas ramas son un inventario histórico; registrar de nuevo las revisiones al efectuar la publicación.

5. Base de datos PostgreSQL y Liquibase

5.1 Preparar almacenamiento local

6. Ubicarse en el repositorio de datos y verificar el contenido del archivo Compose.
7. Crear o seleccionar una base aislada para desarrollo o pruebas; no reutilizar la productiva para ensayos destructivos.
8. Configurar el acceso mediante variables/archivos seguros, sin escribir credenciales en la documentación.
9. Iniciar PostgreSQL por Compose solo después de revisar servicios, puertos y volúmenes.
10. Comprobar que el servicio está saludable y permite conexiones del backend autorizado.

```
cd C:\lavarapido-base-datos
docker compose config --services
docker compose up -d postgres
docker compose ps
```

El nombre de servicio “postgres” debe confirmarse con docker compose config --services. Si difiere, utilizar el nombre real en el comando de arranque.

5.2 Migrar el esquema

11. Determinar si es instalación inicial o actualización; documentar respaldo y procedimiento de recuperación antes de aplicar cambios.
12. Identificar la raíz aprobada: changelog-master.yaml y changelog-production.yaml aparecen en documentación técnica; no intercambiarlas sin revisar su contenido.
13. Ejecutar validate y status con la configuración del ambiente.
14. Aplicar update solamente con autorización y contra la base seleccionada.
15. Registrar DATABASECHANGELOG, checksums y presencia de restricciones esenciales.

```
liquibase validate
liquibase status
# Tras aprobación y con la configuración correcta:
```

```
liquibase update
```

Los comandos Liquibase requieren parámetros de conexión y archivo de changelog suministrados por el ambiente o su configuración. No ejecutar update contra producción sin respaldo y ventana de cambio.

Verificar la presencia de las 18 tablas declaradas, incluidas notificaciones, dispositivos_push y notificacion_entregas; comprobar unicidad de pagos/asignaciones/calificaciones por reserva, referencias Wompi y exclusión de solapamiento de reservas en el DDL aplicado.

6. Backend Spring Boot y seguridad

6.1 Configuración

16. Confirmar Java 17, propiedades por perfil y disponibilidad de PostgreSQL.
17. Configurar URL/usuario/clave de DB, secreto JWT, CORS, URL de frontend y opciones de proveedores mediante mecanismos seguros.
18. Mantener spring.jpa.hibernate.ddl-auto=validate si es la opción declarada; el backend no reemplaza a Liquibase para migrar el esquema.
19. Compilar y ejecutar pruebas automatizadas en un ambiente que permita esas operaciones, separando evidencias por fecha y revisión.
20. Iniciar la API con el perfil autorizado y comprobar las rutas HTTP públicas y protegidas.

```
cd C:\backend\backend-vehicular
.\mvnw.cmd test
.\mvnw.cmd package
# Ejecución local (perfil y variables configurados):
.\mvnw.cmd spring-boot:run
```

No usar valores de credenciales en la línea de comandos ni capturas. Para ambientes Linux, utilizar ./mvnw en lugar de .\mvnw.cmd. La ejecución puede requerir conexión a PostgreSQL y dependencias previamente descargadas.

6.2 Verificaciones de API

Verificación	Resultado esperado	Evidencia
GET /actuator/health/readiness	HTTP 200, UP cuando el servicio está listo	Fecha, código HTTP y salida sin secretos
Acceso sin JWT a endpoint protegido	Respuesta de acceso denegado	Petición y HTTP de respuesta

JWT válido con rol permitido	Acceso al recurso autorizado	Prueba con datos ficticios
Acceso a recurso ajeno	Rechazo controlado	Caso de autorización
Servicio/reserva/pago	DTO y estado conforme al contrato	Prueba funcional en QA

La ruta de readiness respondió HTTP 200 y UP en una verificación histórica de octubre de 2026; el health general devolvió HTTP 503 por dependencia SMTP. Esos resultados no prueban el estado actual del backend.

7. Integraciones: Wompi, correo y notificaciones

7.1 Pagos Wompi

21. Definir ambiente sandbox y configurar las variables requeridas en backend.
22. Crear una reserva apta con datos sintéticos y solicitar inicio del intento de pago.
23. Abrir el widget del proveedor con la información pública entregada por la API.
24. Validar el resultado con consulta servidor-servidor o webhook firmado; comparar referencia, monto, identificador, ambiente y moneda.
25. Confirmar estados pendiente/aprobado/rechazado y manejo de intentos duplicados sin doble aprobación.

La clave privada de proveedor y los secretos de eventos pertenecen exclusivamente a la configuración del backend. No se debe asumir que el retorno del widget es una aprobación definitiva.

7.2 Recuperación y notificaciones

- Correo: configurar SMTP mediante variables de entorno; probar solicitud, expiración y uso único de enlaces en un ambiente controlado.
- Notificaciones: comprobar generación y consulta dentro de la app, contador no leído y marcado como leído; las entregas Expo Push/correo dependen de configuración y habilitación del ambiente.
- El despliegue no se considera conforme si una integración crítica falla sin manejo de errores y sin evidencias de su estado real.

8. Panel administrativo React

26. Entrar al directorio del panel y revisar package.json y lockfile.
27. Instalar dependencias de forma reproducible con npm ci.

28. Configurar la URL de la API por el mecanismo Vite del ambiente; comprobar el prefijo /api sin duplicarlo.
29. Ejecutar los scripts de pruebas, typecheck y build que existan realmente en package.json.
30. Publicar el directorio resultante de Vite en el hosting elegido y configurar fallback SPA para rutas profundas.
31. Comprobar acceso a login y dashboard, validación ADMIN mediante RequireAdmin y tratamiento de 401/403.

```
cd C:\Frontend-lavarapido\lava-rapido-admin
npm ci
npm run
npm run build
```

El comando npm run permite verificar los scripts disponibles antes de ejecutar pruebas opcionales. No asumir que existe un script npm run typecheck si no aparece en el manifest.

9. Aplicación móvil Expo y React Native

32. Entrar al proyecto móvil y ejecutar npm ci con el lockfile vigente.
33. Inspeccionar app.json, app.config.js y eas.json antes de seleccionar la distribución.
34. Configurar EXPO_PUBLIC_API_URL con el backend correspondiente y comprobar HTTPS en URLs públicas.
35. Ejecutar scripts disponibles de validación (npm run) y iniciar Expo en desarrollo.
36. Para distribución Android utilizar el perfil EAS autorizado; registrar ID, versión, fecha y enlace del artefacto sin compartir credenciales.
37. Instalar el APK de prueba en un dispositivo compatible y verificar navegación de cliente y operador, permisos y comunicación API.

```
cd C:\Frontend-lavarapido\lava-rapido-mobile
npm ci
npm run
npx expo start
# Solo si existe perfil preview válido y EAS configurado:
eas build --platform android --profile preview
```

La aplicación declara Expo SDK 58 en el manifest examinado el 4/11/2026. No confundir este estado con builds históricos de Expo SDK 57. Toda evidencia de instalación debe asociarse a una build concreta.

10. Procedimiento de despliegue y publicación

10.1 Orden operativo

38. Respaldar y validar PostgreSQL, junto con la raíz Liquibase que corresponda al ambiente.
39. Aplicar y comprobar migraciones aprobadas antes de publicar código que dependa del nuevo esquema.
40. Configurar variables y desplegar backend; validar readiness y conectividad según la política de salud.
41. Construir/publicar panel React con origen API autorizado y comprobar navegación SPA.
42. Generar/distribuir compilación móvil EAS y validar su conexión con la API publicada.
43. Ejecutar casos de aceptación con datos ficticios: login, catálogo, vehículos, reservas, asignaciones, estados y notificaciones.
44. Ensayar pago solo en ambiente sandbox; revisar recuperación de contraseña y fallos de proveedor.
45. Registrar revisiones, resultados y decisiones de aceptación o reversión.

10.2 URLs públicas reportadas

Componente	Dirección reportada	Alcance de la evidencia
Backend	https://lava-rapido-backend.onrender.com	URL documentada; verificación histórica de readiness en octubre
Panel web	https://lava-rapido-admin.onrender.com	URL comunicada para el panel; respuesta actual no comprobada
Android	Distribución mediante Expo/EAS	La URL/ID de build vigente debe anexarse para esta versión

Durante la elaboración de este documento no fue posible acceder desde la consulta disponible a las URLs públicas; no se afirma que respondan actualmente ni se adjuntan capturas supuestamente tomadas de Render. Las capturas previas del panel y la app son evidencia de interfaces, no constancia inequívoca de despliegue remoto.

10.3 Registro mínimo de publicación

Campo	Información que debe anotarse
Ambiente y fecha	Desarrollo, QA o producción; hora y zona horaria
Versiones	SHA de frontend, backend y datos; versión EAS si aplica
Servicio/URL	Nombre del servicio y dirección pública
Resultado	Código HTTP / captura de consola sin datos reales
Operación de datos	Changelog y estado de aplicación; respaldo

Responsable	Rol autorizado y decisión de aceptar/revertir

11. Verificación posterior y evidencias

La evidencia debe permitir reconstruir qué versión se publicó, dónde se comprobó y cuál fue el resultado. Los registros y las capturas compartidas deben eliminar nombres, correos, placas, tokens, claves y datos de pagos de personas reales.

ID	Evidencia que se debe adjuntar	Situación documentada
EV-01	Panel de hosting con servicio, revisión y estado, datos privados ocultos	Por adjuntar para publicación vigente
EV-02	GET readiness con fecha/hora, HTTP y cuerpo depurado	Antecedente octubre 2026: 200 UP
EV-03	Comprobación HTTPS y carga del panel	URL reportada; comprobación vigente pendiente
EV-04	DATABASECHANGELOG / validación de migraciones	No disponible de instancia actual
EV-05	EAS build y prueba de instalación Android	Requiere build/instalación asociados a versión
EV-06	Resultados de pruebas de autenticación, reserva y pago sandbox	Requiere evidencia por ambiente
EV-07	Capturas del panel y móvil anonimizado	Material visual aportado; prueba interfaz, no hosting

11.1 Prueba de aceptación mínima

Caso	Secuencia resumida	Resultado esperado
IP-01	Acceso administrativo a dashboard	ADMIN autorizado; perfiles restantes rechazados
IP-02	Registrar vehículo y reserva en horario	Reserva PENDIENTE sin solapamiento
IP-03	Asignar operador e iniciar/finalizar	Transiciones consistentes y consulta autorizada
IP-04	Pago sandbox aprobado/rechazado	Estado aplicado tras verificación del backend
IP-05	Consultar/marcar notificación	Historial y contador consistentes
IP-06	Simular expiración de sesión	Ruta protegida rechaza token inválido

Los resultados de estas pruebas son criterios de aceptación: no se marcan como aprobados mientras no haya evidencia de ejecución asociada a la revisión liberada.

12. Gestión de incidencias y reversión

12.1 Respuesta a fallas

- Backend no inicia: revisar versión Java, perfil, variables, conexión a datos y estado de migraciones sin revelar secretos en logs.
- Readiness falla: verificar dependencias críticas y distinguir liveness, readiness y health general; documentar la respuesta exacta.
- Panel no carga rutas internas: revisar configuración de fallback de SPA y dirección pública de API.
- Pago no confirma: mantener estado pendiente/rechazado según reglas y consultar Wompi desde backend; no forzar aprobación local.
- Migración falla: detener liberación y aplicar plan de restauración probado; no reescribir changesets ya aplicados.

12.2 Plan de reversión

46. Detener nuevas publicaciones y registrar incidente, ambiente y revisiones afectadas.
47. Restablecer la compilación anterior del backend/panel y confirmar compatibilidad con el esquema.
48. Restaurar la base únicamente con respaldo autorizado y evaluación de datos generados desde el cambio.
49. Verificar readiness, autenticación, operaciones esenciales y trazabilidad tras la recuperación.
50. Cerrar con acta de incidencias y acciones preventivas.

13. Lista de comprobación de entrega

Control	Criterio de cierre	Estado con evidencia disponible
Versiones	Commits y builds identificados	Versiones declaradas; revisión publicada pendiente
Base	Schema y changelog aplicados / respaldo probado	No confirmado en instancia desplegada
API	HTTPS, permisos y readiness	Readiness histórico disponible
Web	Panel accesible y rutas SPA funcionales	URL documentada; validación vigente pendiente
Móvil	Instalación y API verificadas en build actual	Requiere acta/artefacto de build actual
Wompi	Sandbox y webhooks verificados	No demostrado por fuentes actuales
Correo/Push	Envíos y fallos controlados	Requiere prueba en ambiente habilitado
Documentación	Manual, SRS, diseño y registro de incidencias	Material académico disponible; evidencias faltantes identificadas

14. Conclusiones y referencias

El proyecto cuenta con los componentes y configuraciones necesarias para seguir un procedimiento de implementación ordenado: PostgreSQL con Liquibase, backend Spring Boot con control de acceso, frontend administrativo y aplicación móvil Expo. La implementación debe avanzar desde el esquema hacia la API y después los clientes, registrando versiones y verificaciones en cada entorno. Las URLs públicas y las configuraciones de despliegue documentadas sirven como punto de partida, pero la aprobación final requiere evidencias actuales de publicación, migración, seguridad e integración.

Referencias

Equipo de Lava Rápido Vehicular. (2026). Especificación de requisitos de software (SRS) [Documento interno].

Equipo de Lava Rápido Vehicular. (2026). Documentación de diseño del software [Documento interno].

Equipo de Lava Rápido Vehicular. (2026). Procedimiento de implementación del software [Documento interno, versión previa].

Equipo de Lava Rápido Vehicular. (2026). Informe técnico de pruebas de rendimiento con Apache k6 [Documento interno].

Liquibase. (s. f.). Documentation. https://docs.liquibase.com/

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/

Expo. (s. f.). Expo documentation. https://docs.expo.dev/

Render. (s. f.). Render documentation. https://render.com/docs

Wompi. (s. f.). Documentación para desarrolladores. https://docs.wompi.co/